

Business Logic Layer (BLL) Report

Q1. Core BLL Modules & Interaction with Presentation Layer

1.1 Core BLL Modules

Module	Responsibility
Authentication Module	Handles login, role validation (student/teacher), and session management.
Essay Submission Module	Accepts essay uploads from students, validates format and length.
AI Review Module	Invokes NLP model to score and generate feedback on submitted essays.
Manual Review Module	Allows teachers to view, annotate, and score essays manually.
Score Aggregation Module	Merges AI score and teacher score; computes the final grade.

1.2 BLL–Presentation Layer Interaction

The presentation layer communicates with the BLL exclusively via REST API endpoints. No business logic resides in the UI. Below is the interaction flow:

UI Action	BLL Module Called	Response to UI
Student clicks 'Submit Essay'	Essay Submission Module	Submission ID, status confirmation
Student clicks 'Get AI Review'	AI Review Module	Score + structured feedback object
Teacher opens essay dashboard	Manual Review Module	List of pending essays with metadata
Teacher submits manual score	Score Aggregation Module	Final score computed and returned
Student views results page	Score Aggregation Module	Combined score + review breakdown
Login form submitted	Authentication Module	JWT token / session cookie

All API calls follow the pattern: UI → REST Endpoint → BLL Module → DAL → Response. The UI never directly queries the database.

Q2. BLL Description — Business Rules, Validation & Data Transformation

A) Business Rules Implementation

Business rules are enforced in the BLL before any data is persisted or returned to the UI. The following rules are implemented per module:

Authentication & Access Control

- Only registered users with verified credentials can log in.
- Role-based access: students can only submit and view their own essays; teachers can view all essays assigned to them.
- A student cannot access the teacher dashboard and vice versa — enforced via JWT role claims checked at every protected route.

Essay Submission

- A student can only submit one essay per assignment. Re-submission is blocked until the teacher explicitly enables it.
- Essays must meet a minimum word count (e.g., 100 words) and maximum word count (e.g., 2000 words) defined per assignment.
- Submission is only allowed within the deadline window set by the teacher.

AI & Manual Review

- AI review is triggered automatically upon successful essay submission.
- Teachers can override or supplement the AI score but cannot delete the AI-generated feedback log.
- A final score is only published to the student after the teacher has reviewed and confirmed it.

Score Aggregation

- Final Score = (AI Score × weight) + (Teacher Score × weight), where weights are configurable per assignment.
- If the teacher chooses to override AI score entirely, the teacher score is used as the final score.

B) Validation Logic

Validation is implemented at two levels: client-side (lightweight UX validation) and server-side BLL (authoritative validation). Server-side validation is always enforced regardless of client state.

Field / Action	Validation Rule	Error Response
Login credentials	Email format check; password min 8 chars	400 Bad Request with field-level error
Essay text	Min 100 words, max 2000 words; non-empty	422 Unprocessable Entity
File download	Accepted types: .txt, .docx, .pdf; max 5MB	415 Unsupported Media Type
Teacher score input	Score must be numeric, within 0–100 range	400 Bad Request
Duplicate submission	Check existing essay by (studentId, assignmentId)	409 Conflict

All validation errors are returned as structured JSON objects with error code, field name, and a human-readable message, which the UI renders inline next to the relevant form field.

C) Data Transformation

Raw data retrieved from the database is transformed by the BLL before being sent to the UI. The UI receives only clean, formatted, UI-ready objects — never raw DB rows.

Key Transformations

Data Source	Raw Format	Transformed Output for UI
Essay submission timestamp	Unix epoch integer (DB)	Human-readable: 'Apr 5, 2026 10:30 AM'
AI feedback JSON	Raw NLP model output with nested scores	Flat object: { grammar, coherence, content, total }
Score fields	Separate AI score + teacher score in DB	Merged: { aiScore, teacherScore, finalScore, method }
User role enum (0/1)	Integer flag in DB	String: 'student' or 'teacher' for UI routing
Essay word count	Computed on raw text at submission	Returned as metadata: { wordCount, charCount }
Review status code	Enum: 0=pending, 1=ai_done, 2=published	String label: 'Pending', 'AI Reviewed', 'Graded'

Transformation logic is centralized in dedicated serializer/mapper functions within the BLL. This ensures the DAL and UI are fully decoupled — changes to the database schema only require updates to the mapper, not to the UI components.

Example: Essay Result Object (transformed)

```
{ "essayId": "e123", "submittedAt": "Apr 5, 2026", "wordCount": 450, "aiScore": 74, "teacherScore": 80, "finalScore": 77, "status": "Graded", "feedback": { "grammar": 85, "coherence": 70, "content": 75 } }
```